LegalLens

A Private, Fully On-Device Legal Document Triage Agent

Complete Solution Documentation

ZeroDay OSS Hacktoberfest 2026, IIIT Dharwad

Problem Statement PS2: Agentic AI — Private Legal Document Triage Agent

Team TH3_B0Tz · October 2026 · v1.1

LegalLens is for preliminary document understanding and issue spotting only. It is not a lawyer, gives no legal advice, and never decides whether a clause is valid, legal or enforceable.

1 Executive Summary

LegalLens reads a rental, employment or service agreement **on the user's own machine**, explains clauses in plain language, and flags obligations, penalties, deadlines, termination terms and ambiguities. Every claim is tied to a quoted passage from the document and checked by deterministic code. When the document does not say something — or the question needs a lawyer — LegalLens says so instead of guessing. Nothing the user uploads ever leaves the device.

The system is built on one design rule: **the model proposes, code disposes.** A local edge-class Gemma model drafts explanations as structured JSON with verbatim quotes; plain code then verifies every quote against the source document, every number and date in every explanation, and even the summary itself. Verdict questions such as “is this enforceable?” are refused by a code classifier before the model is ever called. Privacy is measured, not promised: during a full run an in-process network guard and an OS connection monitor recorded zero connections leaving the machine, no document text turned up in any file on disk, and a dependency audit found network code only in the loopback model client.

Item	Value
Track / problem statement	PS2 — Agentic AI: Private Legal Document Triage Agent
Core model	Gemma 4 E4B (edge-class), 4-bit Q4_K_M GGUF, 4.98 GB
Runtime	llama.cpp llama-server on 127.0.0.1 (loopback only); all layers on the demo laptop's RTX 4060
Language / deps	Python 3.11+, pinned requirements

Item	Value
Retrieval	Hybrid BM25 + all-MiniLM-L6-v2 (ONNX) in a FAISS index, in memory
Privacy proof	3 layers: offline run (demo laptop) + monitored run (PASS) + dependency audit (clean)
Licence	MIT (project code); model per its official card
Status	Pipeline, ingestion, retrieval, verification, safety and privacy tooling done (137 tests in CI); UI, evaluation and demo in progress

2 Problem Statement and Scope

2.1 The problem

Legal documents such as rental agreements, employment contracts, legal notices and service agreements contain complex terminology and clauses that ordinary people find hard to understand, and professional help for every first question is not always practical. Generic chatbots may invent clauses, answer from general knowledge and sound certain — and uploading a private contract to a cloud tool is itself a privacy risk.

2.2 In scope

- Languages: English.
- Formats: text-based PDF and plain text (.txt), one file at a time.
- Document types tested: rental / lease agreements and employment contracts (service agreements if time allows).
- Outputs: summary with citations, prioritised findings (obligations, rights, deadlines, penalties, termination, ambiguities, risks), clause-by-clause explanations with the source passage beside each, document-grounded Q&A, and an escalation panel.

2.3 Out of scope (stated honestly)

- OCR for scanned PDFs, multilingual explanations, two-version comparison, in-PDF highlighting (all optional per the rules; never penalised if absent).

2.4 Explicit non-goals

- Judging validity or enforceability; naming applicable laws as conclusions; predicting court outcomes; recommending “sign” or “don't sign”.
- Any cloud call during document processing.

3 Technology Stack

Layer	Choice	Why
Model	Gemma 4 E4B (edge-class), 4-bit Q4_K_M GGUF (4.98 GB)	Strongest edge-class variant that fits the demo laptop's 8 GB GPU entirely; licence per the official model card. Fallback: E2B
Runtime	llama.cpp llama-server on 127.0.0.1, --jinja, thinking off	Local inference, no cloud; the chat endpoint applies Gemma's template; reasoning mode off so answers come back as plain JSON
Language	Python 3.11+, pinned requirements	Reproducibility; CI on Ubuntu and Windows, Python 3.11 and 3.13
Ingestion	pypdf first, pdfplumber for weak pages	Local PDF/.txt; headers and footers stripped; clause-aware chunks

Layer	Choice	Why
		with stable IDs (p3.c2)
Retrieval	BM25 (rank_bm25) + all-MiniLM-L6-v2 run as ONNX by onnxruntime (no PyTorch)	Hybrid: exact legal terms + paraphrase matching, all local
Vector index	FAISS, in process memory only	Nothing written to disk; dropped by clear_cache()
UI	Gradio on 127.0.0.1, analytics off, system fonts, no CDN (in progress)	Source passage shown beside every explanation
Testing	137 deterministic pytest tests; CI	Green without model weights or network

Telemetry kill-switches (set in code, not just documented)

```
HF_HUB_OFFLINE=1   TRANSFORMERS_OFFLINE=1   HF_HUB_DISABLE_TELEMETRY=1   DO_NOT_TRACK=1
GEMMA_BASE_URL is refused unless it points at 127.0.0.1 / localhost
GRADIO_ANALYTICS_ENABLED=False    # for the UI, in .env.example
```

4 System Architecture

Six stages, all on the user's device. **Design rule: the model proposes, code disposes.** Gemma reads and explains; deterministic code checks every quote, every number and every refusal rule. The model never certifies its own output.

#	Stage (lane)	What happens
1	INGEST (B)	pypdf / pdfplumber extract text; running headers and page numbers removed; clause-aware chunker splits on numbered clauses (4., 4.1, Clause 5:, Article IV) and capitalised headings; passages get stable IDs (p3.c2); one clear warning for empty, damaged, encrypted or scanned files
2	INDEX (B)	For questions: BM25 plus all-MiniLM-L6-v2 embeddings in a FAISS index, held in memory only
3	ANALYSE (A)	Gemma on 127.0.0.1 only; document text enters prompts as DATA inside delimiters, never instructions; output is JSON findings (kind, plain_text, quote, passage_id, label, confidence)
4	VERIFY (C)	Deterministic code, no model: quote must be a substring of the cited passage (after normalisation); near-match ≥ 0.95 repaired and flagged; numbers/dates in an explanation must appear in its quote; failures dropped or marked unverified
5	SAFETY (C)	Labels (STATED / INTERPRETATION / CONCERN) with code-side downgrade; ambiguity and jurisdiction flags; “document is silent”; verdict refusal in code; escalation notes; wording filter; severity ordering; fixed disclaimer
6	UI (D, in progress)	Gradio on 127.0.0.1 only: overview with questions to ask before signing · findings with the source passage beside each explanation · Q&A · list for a lawyer

Runtime network policy

Network at normal run time: only LegalLens to the local model on 127.0.0.1. Setup-time downloads only (packages, the Gemma weights, the embedding model): explicitly allowed by the rules, since no document content is involved.

5 Grounding: Evidence on Every Claim

Grounding carries a 15-point criterion and a score cap. Six mechanisms deliver it:

1. **Passage IDs everywhere.** The model only ever sees passages with IDs and must cite one.

2. **Verbatim quote check.** After normalisation (Unicode NFKC, whitespace collapse, curly-quote/dash unification, case-fold), the quote must be a substring of the cited passage. Near-match (≥ 0.95 similarity, e.g. a dropped comma) is repaired with the true source span and flagged quote_corrected; anything else is dropped and counted in eval as a verifier rejection.
3. **Number/date consistency.** Every amount, day-count, percentage and date in an explanation must appear in its quote. Catches "30 days" explained as "one month" — the most harmful plain-language distortion.
4. **"The document is silent."** If an answer's citations do not verify against the retrieved clauses, the reply is the fixed text "The document does not appear to say anything about this", never general knowledge.
5. **Coverage report.** Clause types found and not found ("no termination clause found"), so users see what is missing, not just what is present.
6. **The summary is verified too.** The summary is generated from verified findings only; every sentence carries passage IDs; a sentence whose numbers do not match its citation is dropped, not repaired. The most claim-dense artifact on screen is never the unverified surface.

6 Uncertainty, Refusal and Escalation

6.1 Separating text from interpretation

Label	Meaning	UI treatment
STATED	The quote directly and fully supports the explanation, zero inference	Plain, quote beside it
INTERPRETATION	A reasonable reading that needs inference beyond the literal words	Marked "our reading"; never used for a verdict
CONCERN	Something that may deserve attention (one-sided, vague, unusual)	Marked "worth asking about", with a question to ask

The prompt asks Gemma to pick the label, and code downgrades it when evidence is weak — a finding whose explanation adds a fact not in its quote cannot be STATED.

6.2 Verdict refusal — in code, not the prompt

Questions like "is this legal?", "is this enforceable?", "should I sign?" are classified by a deterministic classifier and refused before the model is ever called. The refusal is specific: which clause, what to ask a qualified legal professional, what facts to bring. Prompt wording is only a backstop for phrasings the classifier misses.

> *"I cannot determine that. This depends on your jurisdiction and specific circumstances. Consider asking a qualified legal professional about <topic>, and bring the exact clause text with you."*

6.3 Ambiguity and jurisdiction

- Vague terms ("reasonable", "sole discretion", "from time to time"), blank placeholders and conflicting clauses become ambiguity findings with flags.
- Jurisdiction-dependent topics (deposit size and return, notice periods, non-compete, probation, overtime, stamp duty, rent increase) are flagged "this depends on where you are" with no legal conclusion.

6.4 Prompt-injection defence

7. Document text enters prompts only inside clearly delimited data blocks, with a system instruction that its contents are untrusted text.
8. A pre-scan flags instruction-like strings (“ignore previous instructions”, “you are now”, “output the following”) and surfaces them: “this document contains text that looks like an instruction to an AI; it was ignored”.
9. Output is a fixed JSON schema — injected text cannot add fields or change the response shape.
10. Verdict refusal happens before the model is called, so an injected instruction cannot talk the system into a legal conclusion.

Tested with an injected sample document: the analysis output contains no trace of the injected instruction, and the user is warned (asserted in the test suite).

7 Retrieval: BM25 + Embeddings + FAISS

7.1 Why retrieve at all

Documents run 30+ pages; the 8k context cannot hold them, and long contexts make small models paraphrase instead of quote. At upload the document is indexed once; per question only the top-3 clauses enter the prompt. Retrieval is the funnel that makes both grounding and the ~30-second latency budget possible.

7.2 Two retrievers that fail in opposite ways

	BM25	Dense embeddings
What it is	Classic exact-word ranking (rare-word weighting, length normalisation)	all-MiniLM-L6-v2 maps clauses and the question to vectors; nearest vectors win
Best at	Precise legal terms: “security deposit”, “non-compete”, “lock-in period”	Paraphrases: “can I get my money back if I leave early?” → deposit-refund clause
Fails when	User's words don't match the document's words	Rare exact legal terms a small embedding model under-saw
Cost	Pure math over word counts — deterministic, instant, no download	91 MB ONNX model downloaded at setup; ~4 ms per query on CPU

Why both: they fail in opposite ways. Legal language is keyword-heavy; user questions are paraphrase-heavy. Rankings are merged (reciprocal-rank fusion) and the top-3 passages go to the prompt. A small lay-term glossary expands the BM25 query only (“rent a room to a friend” → sublet). Measured: the right clause is in the top 3 for 15 of 15 handwritten questions over both sample contracts.

7.3 FAISS — the local vector index

FAISS (Facebook AI Similarity Search) stores the clause vectors and returns the top-k nearest in milliseconds — in-process, open source, no network. For a 3-page rental a brute-force scan would do; FAISS costs nothing and serves unbounded documents with the same code path. Similarity only: all semantic quality comes from the embedding model; FAISS is pure speed.

7.4 The silent path

Silence is decided by verification: if an answer's citations do not check out against the retrieved clauses, code returns “the document does not appear to say anything about this”. Retrieval also scores relevance, and unanswerable questions score low (about 0.2 against 0.5), so a threshold that skips the model call is possible; it is not enabled yet.

8 Model Choice: Why This Gemma Variant

The rules mandate an edge-class Gemma running on-device doing meaningful document understanding — either generation (Gemma 3n or Gemma 4 E2B/E4B) is accepted. The variant was derived from these constraints:

Constraint	Rules out	Survives
Rules: edge-class Gemma, local, private	9B/27B+; any cloud inference	E2B / E4B
Hardware: demo laptop with an RTX 4060 (8 GB VRAM) and 24 GB RAM	Weights over ~6.5 GB on the GPU	E4B 4-bit, 4.98 GB, entirely on the GPU
Task: character-exact quotes, JSON schema, refusals	Weak instruction-following → more verifier rejections	E4B over E2B
Latency: interactive Q&A	Big prompts / models	top-3 retrieval + E4B on the GPU

- Why Gemma 4 generation: the same accepted edge class, with good JSON reliability; licence terms per the official model card.
- Why 4-bit (Q4_K_M): RAM/quality sweet spot for extraction-style tasks.
- **Fallback ladder (measured, not guessed):** context 8192 → 4096 → Q3_K_S → E2B. Measured so far on a CPU-only laptop (about 5 tokens/s): 11 of 11 findings verified on the sample rental, 110–265 s per full analysis, 15–30 s per question. GPU numbers to be recorded on the demo laptop.

9 Privacy Architecture and Proof

Privacy carries a 15-point criterion and a score cap. Judges look for the absence of connections (encrypted payloads are invisible), so the proof needs three independent layers:

Layer	Claim	Method
1	Cannot connect	run_offline.ps1 refuses to start while the machine is online, records the network state, then runs the monitored capture in Airplane mode (Linux: run_offline.sh in a network namespace). To be recorded on the demo laptop
2	Does not try	Recorded, PASS. Python audit hooks record and block every connect, sendto, DNS lookup and urllib call; an OS monitor watches LegalLens and llama-server; a canary string from the document is searched for in every file touched. Full run with retrieval: 12 attempts, all to 127.0.0.1:8080, none outside; canary in none of 17 files
3	Has no code that connects	Recorded, clean. privacy/audit_deps.py scans our code and 43 runtime dependencies and profiles a real session: network code only in our loopback model client; 4 dormant numpy hits never called

9.1 Supporting controls

- The model server binds to 127.0.0.1; the retrieval index runs in-process, in memory; the model URL is refused unless it points at this machine.

- Weights and the embedding model are setup-time downloads, loaded from disk at runtime; HF_HUB_OFFLINE=1 and friends are set in code, making hub access impossible even by accident.
- Document-derived data stays in memory; the canary check confirms nothing document-derived lands on disk, the llama-server log included.
- Logs hold timings, counts and error codes only. Full detail and every network call: privacy/THREAT_MODEL.md.

9.2 Threat model summary

- **Protected:** the file, extracted text, chunks, embeddings, summaries, answers, names, metadata, and the user's questions.
- **Defended against:** remote services, telemetry, shared-machine leakage, instruction injection inside documents.
- **Not claimed:** a compromised OS, a malicious model file, physical or screen access.

10 Module Reference

Core types (legallens/types.py)

```
Passage:   id ("p3.c2"), page, heading, text, start, end
Document:  doc_id, source_name, passages, extraction_warnings
Finding:   id, kind, party, plain_text, quote, passage_id, label,
           confidence, flags, escalation_note
  kind:  obligation | right | deadline | penalty | termination | ambiguity | risk
  label: STATED | INTERPRETATION | CONCERN
Analysis:  summary, summary_citations, findings, coverage
Reply:     text, citations[{passage_id, quote}], silent, refused, escalation_note
VerifyResult: ok, reason, repaired_quote, pruned_summary
```

Function signatures

```
# Lane A
analyze(doc: Document) -> Analysis          # extract -> verify -> summary -> coverage
ask(doc: Document, question: str) -> Reply  # route in code -> retrieve -> answer -> verify

# Lane B
ingest(path) -> Document                 # never raises on bad files
retrieve(doc, query: str, k=3) -> list[tuple[Passage, float]]

# Lane C
verify(item: Finding | Reply | Analysis, doc) -> VerifyResult
classify_question(q) -> explain | document_question | verdict_request | off_topic
prioritise(findings) -> list[Finding]      # most severe first

# Lane D (planned)
load_sample(name: str) -> str
eval_set() -> list[EvalCase]
```

Module boundaries are import boundaries in analyze.py. Every module has landed; the fallbacks remain only so each lane stays testable on its own and CI runs without model files.

11 Test and Evaluation Plan

11.1 Current test suite (137 passing, no model or network needed)

- JSON client handling: fences, chatty replies, top-level arrays, retries with error feedback, clear errors for a dead server, loopback-only configuration.
- Pipeline: findings, summary citations, coverage, severity order, and graceful handling of bad JSON, timeouts and partial results.
- Ingestion: clause splitting, headings, header/footer stripping, stable IDs and offsets, and malformed files (empty, damaged, password-protected, scanned).
- Retrieval: hybrid ranking on 15 handwritten questions, BM25-only fallback, and the retrieved clauses reaching the prompt.
- Grounding and safety: fabricated quotes and changed numbers rejected, label downgrade, refusal, the wording filter, injection.
- Privacy: the network guard blocks outside connections and DNS but allows loopback; the audit finds only the model client.
- Safety paths: verdict questions refused in code (model never called — asserted via empty call log), “should I sign” refused, silent path, injected document contained (no “fully legal” leak).

11.2 Still to do (lane D)

- Evaluation on documents nobody tuned against: citation accuracy, silent-case and refusal-case correctness, ambiguity flagging, injection results, human-scored meaning accuracy.
- More samples: vague and conflicting contracts, an injected PDF, a 30+ page document, and 1–2 public templates.
- Speed and memory on the demo laptop (RTX 4060), reported with exact hardware.

11.3 Metrics reported honestly

Quote-verification pass rate before and after repair; findings dropped; citation accuracy; silent-case and refusal-case correctness; label distribution; latency per document and per question; peak RAM; model size and quantisation; known failures with examples — reporting weaknesses scores better than hiding them.

12 Implementation Status

Lane	Scope	Status
A — Gemma + analysis pipeline	llm.py, prompts/, analyze.py: local client, JSON extraction with retries, summary with citations, Q&A, failure handling	Done; real-mode verified
B — ingestion, retrieval, privacy	ingest.py, retrieve.py, privacy/: parsing, chunking, hybrid index, network guard, monitors, audit, threat model	Done; offline run laptop
C — grounding + safety	verify.py, safety.py, injection.py, priority.py: verifier, labels, flags, refusal, escalation, wording filter, severity	Done
D — UI, bench, docs, demo	app/, bench/, data/samples/, docs, CI, demo recording, submission	In progress: UI d CI done; Gradio a video to come

13 Honest Limitations

11. English only; text-based PDF and .txt; no OCR for scans (detected and reported, never guessed).
12. Tested scope: rental and employment agreements we wrote; evaluation on unseen real-world templates is still to do.
13. Retrieval bounds Q&A quality — a relevant clause ranked low yields a silent answer, never a wrong one. A deliberate trade: silence is honesty; a wrong citation is fabrication.
14. Severity is a fixed formula, so a mutual notice clause can rank as critical; the weights are under review.
15. Small-model wording is kept deliberately terse; the number/date check hides any explanation that drifts.
16. Privacy covers the software, not a compromised OS or a malicious model file — scope stated in the threat model.

14 Main Technical Contribution

> *"A verification-first architecture for small local models: we don't trust Gemma to be right, we make it provably grounded — quotes, numbers and the summary are all checked by deterministic code, so a 4-bit edge model can't fabricate. The refusal logic lives in code, not the prompt, so it can't be talked out of it — not even by instructions hidden in the document. And the privacy claim comes with recorded evidence: a network guard, a connection monitor and a canary leak check."*

Why this matters: the three failure modes the rubric penalises most — fabricated citations, unsupported legal conclusions, and content leaving the device — are blocked by code rather than discouraged by prompting. The verifier and the chunker are reusable components in their own right.

15 How to Reproduce

Setup (needs network once)

```
git clone https://github.com/ZERODAY-2026/TH3_B0Tz.git && cd TH3_B0Tz
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# Gemma 4 E4B Q4_K_M (4.98 GB) and the embedding model (91 MB);
# exact commands and SHA-256s in docs/MODEL_SETUP.md
python -c "from huggingface_hub import hf_hub_download as d; \
  d('unsloth/gemma-4-E4B-it-GGUF', 'gemma-4-E4B-it-Q4_K_M.gguf', local_dir='models')"
```

Run (offline)

```
llama-server -m models/gemma-4-E4B-it-Q4_K_M.gguf --host 127.0.0.1 \
    --port 8080 -c 8192 -ngl 99 --jinja \
    --chat-template-kwargs '{"enable_thinking":false}'

python smoke.py --file data/samples/rental_clear.pdf   # real file, end to end
python smoke.py            # scripted model, no server needed
pytest tests/ -q           # deterministic suite, no model/network
```

The README separates the one-time model download from running. The offline claim is verified with privacy/run_offline.ps1, which refuses to run while the machine is online and records the network state and connection monitor output.

16 Demo Script (~5 minutes)

17. **Problem (30 s):** People sign rentals and jobs without understanding them; generic chatbots invent clauses and sound certain; uploading a private contract to the cloud is a privacy risk.
18. **Privacy proof (60 s):** Disable networking; run the full analysis live; show the connection monitor with zero outbound connections.
19. **Live on an unseen document (2.5 min):** Upload an agreement the team did not tune on. Show summary and top findings with the quoted passage beside each explanation and its label. Ask a question the document cannot answer → "the document does not say". Ask "is this enforceable?" → declined with escalation. Open injected.pdf → instruction flagged and ignored.
20. **Results (45 s):** Eval table: citation accuracy, verifier rejections, silence and refusal cases, speed and RAM on the demo laptop, plus one honest failure.
21. **Close (15 s):** Gemma runs on the device, code checks every quote, nothing leaves the machine, and the tool knows its limits.

17 Licence and Attribution

- Project code: MIT (LICENSE in the repository root).
- Model: Gemma — licence per its official model card (see docs/MODEL_SETUP.md for acquisition steps and the recorded SHA).
- Sample documents: written by the team, safe to redistribute with the project.
- Optional benchmark CUAD (CC BY 4.0): cited in any results table that uses it; never committed to the repository.
- No user documents, credentials or model weights are committed (gates G4, G5).